

# Asincronía

01

# Funciones asíncronas

# Funciones Asíncronas

Las funciones asíncronas en **JavaScript** son aquellas que permiten la ejecución de operaciones no bloqueantes mediante el uso de **promesas** o la estructura **async/await**. Permiten realizar operaciones que pueden llevar tiempo, como *solicitudes de red o lecturas de archivos*, sin detener la ejecución del programa

- Promesas
- Estructura async/await
- Observables

02

## Promesas

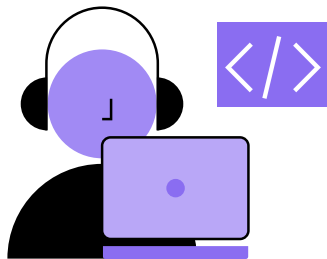
# Promesas:

Una promesa representa un **valor** que puede estar **disponible** *ahora, en el futuro o nunca*. Se utiliza comúnmente para manejar operaciones **asíncronas** como *solicitudes a servidores, lectura/escritura de archivos, entre otras*.

## ESTADOS:

- **Pendiente (Pending)**: El estado inicial cuando la promesa está creada y aún no se ha completado o rechazado.
- **Cumplida (Fulfilled)**: La operación asociada con la promesa se completó con éxito, y el resultado se hizo disponible.
- **Rechazada (Rejected)**: La operación asociada con la promesa falló, y se proporciona una razón para el fallo.

Las promesas proporcionan una forma más legible y manejable de trabajar con código asíncrono en comparación con el uso de callbacks anidados. Además, permiten encadenar operaciones asíncronas de manera más clara mediante el método `.then()`.



```
let miPromesa = new Promise(function(resolve, reject) {
  // Simulando una tarea asíncrona
  setTimeout(function() {
    let exito = true; // Cambia a false para simular un fallo
    if (exito) {
      resolve("¡Éxito!"); // La promesa se cumple
    } else {
      reject("¡Fallo!"); // La promesa se rechaza
    }
  }, 2000); // Simula una operación que lleva 2000 milisegundos (2 segundos)
});

miPromesa.then(function(valor) {
  console.log(valor); // Se ejecuta cuando la promesa se cumple
}).catch(function(razon) {
  console.error(razon); // Se ejecuta cuando la promesa se rechaza
});
```

## 03 Async/Await

# Async/Await:

Son *palabras clave* trabajan en conjunto para simplificar la sintaxis y hacer que el **código asíncrono** se vea como el **código síncrono**. **Async** se utiliza para declarar una función asíncrona y **Await** se utiliza dentro de una función **async** y espera a que una **promesa** se *resuelva* antes de continuar con la ejecución del código

```
async function miFuncionAsincrona() {  
  try {  
    let resultado = await algunaOperacionAsincrona();  
    console.log(resultado);  
  } catch (error) {  
    console.error(error);  
  }  
}
```